

100 Days of Azure DevOps

Day 27 Handout — The Maven train is compile → test → package. Jumping off at compile dumps untested jars on the tracks

3 - Continuous Integration · Personal lab only · Educational content · Not a sales pitch

Architecture A — Maven lifecycle on a hosted job

Phase	What Maven does	CI command
validate / compile	POM + javac against JDK on PATH	mvn -B -f src/sample-java/pom.xml compile
test	Surefire unit tests (fail the build)	mvn -B -f src/sample-java/pom.xml test
package	jar/war after tests (default)	mvn -B -f ... package (not -DskipTests)
JDK pin	JavaToolInstaller@0 versionSpec: '17' PreInstalled	Do not trust 'the agent had Java'
Cache	Cache@2 on \$(HOME)/.m2/repository	Key includes pom.xml hash or you re-download the internet

Architecture B — when to skip a deep Java app

Constraint	Do this	Do not do this
Java is your stack	mvn archetype + green test on main	Skip tests to 'save minutes'
Java is not your stack	Read the YAML + lifecycle; write 8 lines of notes	Skip the day. Interviews still ask about the train
Build too slow	Cache@2 + -B; shrink the module	-DskipTests as a lifestyle
Need a jar for later CD	mvn -B package then PublishPipelineArtifact@1	Compile-only job labeled 'CI'

Architecture C — pitfalls that ship shaded optimism

Pitfall	What you see	Fix
Wrong JDK	illegal start / preview errors; works on laptop 21	JavaToolInstaller@0 17; echo \$JAVA_HOME
cwd folklore	Unable to find pom.xml at /home/vsts/work/1/s	Always -f src/sample-java/pom.xml
Interactive Maven	Job hangs on a prompt nobody will answer	mvn -B (batch mode) in every CI script
-DskipTests	Green in 40s; NPE in the first request	test or verify. Fire exit, not a habit

One-liner to remember

Maven phases are a train: compile → test → package — jumping off early dumps jars on the tracks.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal Azure DevOps org only. Project azure-100-labs. Hosted ubuntu-latest. Deep Java app is optional; the lifecycle is not.

- Option A: from a clean folder run mvn -B archetype:generate (quickstart) into src/sample-java and commit the pom + one unit test.
- Option B (time-box): copy a public tiny pom + test into src/sample-java. Do not import an employer repo.
- Add pipelines/maven-ci.yml with JavaToolInstaller@0 (17, x64, PreInstalled) and mvn -B -f src/sample-java/pom.xml test.
- Pipelines → run the YAML. In the log find JAVA_HOME / Java version and the Surefire summary.
- If you skip a full app, write docs/maven-lifecycle-day27.md: validate, compile, test, package, verify — one sentence each.

6 Do not add -DskipTests. If the job is slow, add Cache@2 on ~/.m2 keyed by pom.xml, not a skip.

Starter YAML — JDK 17 + mvn -B test

```
# pipelines/maven-ci.yml
trigger:
  paths:
    include: [ src/sample-java/** ]
pool:
  vmImage: ubuntu-latest
steps:
- task: JavaToolInstaller@0
  inputs:
    versionSpec: '17'
    jdkArchitectureOption: x64
    jdkSourceOption: PreInstalled
- task: Cache@2
  inputs:
    key: 'maven | "$(Agent.OS)" | src/sample-java/pom.xml'
    path: $(HOME)/.m2/repository
- script: mvn -B -f src/sample-java/pom.xml test
  displayName: Maven test (no -DskipTests)
```

Done checklist

- Can list compile → test → package without calling it 'just build'
- YAML pins JDK 17; mvn uses -B and -f
- No -DskipTests on the lab pipeline
- Notes or a green Surefire run exist on the personal repo

Tomorrow — Day 28

Multi-stage YAML — dependsOn, succeeded(), and a matrix so 'works on my version' becomes two jobs.